# LIVRABLE 0 - Guide de Présentation - Stripe Data Architecture

**Entreprise :** Stripe — FinTech fondée en 2010 à San Francisco.
**Échelle :** Milliards de transactions par an, millions de marchands dans le monde.
**Produit :** Plateforme de traitement des paiements pour les entreprises.

**Objectif du projet :** Concevoir une architecture de données unifiée intégrant OLTP, OLAP et NoSQL pour supporter les opérations transactionnelles, l'analytique avancée et le Machine Learning (détection de fraude, personnalisation client).

---

## Défis Métier

L'expansion mondiale de Stripe génère cinq défis principaux :

- **Transactions :** Les systèmes OLTP traitent des millions de transactions quotidiennes (paiements, remboursements, rétrofacturations). Le système doit garantir haute disponibilité, conformité ACID et détection de fraude en temps réel.
- **Analytique :** Stripe doit fournir analyses de revenus, segmentation client, reporting de conformité et suivi produit via un système OLAP capable de requêtes complexes en quasi temps réel.
- **Données non structurées :** Logs, clickstream, feedback client — ces données alimentent les modèles ML (fraude, personnalisation, prédictif) et nécessitent une base NoSQL flexible.
- **Intégration :** Assurer un flux cohérent entre OLTP, OLAP et NoSQL avec synchronisation temps réel et batch.
- **Conformité :** Architecture conforme GDPR, PCI-DSS et CCPA — chiffrement, contrôle d'accès et journalisation d'audit.

---

## Sources de Données

| Source | Champs clés |
|---|---|
| **OLTP** | Transaction ID, marchand, client, montant, devise, méthode paiement, date/heure, localisation IP, appareil, statut, indicateurs fraude |
| **OLAP** | Métriques revenus (jour/semaine/mois), segmentation client, performance produit, analyse fraude, logs audit |
| **NoSQL** | Logs applicatifs, clickstream/sessions, features ML, feedback client |
| **Référence** | Codes pays/régions, taux de change, infos marchands, catalogues produits |

---

## Livrables

| # | Livrable | Fichier |
|---|---|---|
| 1 | Diagramme d'architecture complet | `1-DIAGRAMME_ARCHITECTURE.md` |
| 2 | ERD du système OLTP | `2-DIAGRAMME_ERD_OLTP.md` |
| 3 | Schéma OLAP | `3-DIAGRAMME_OLAP.md` |
| 4 | Modèle NoSQL | `4-MODELE_DONNEES_NOSQL.md` |
| 5 | Sécurité et conformité | `5-SECURITE_CONFORMITE.md` |
| 6 | Intégration ML | `6-STRATEGIE_INTEGRATION_ML.md` |

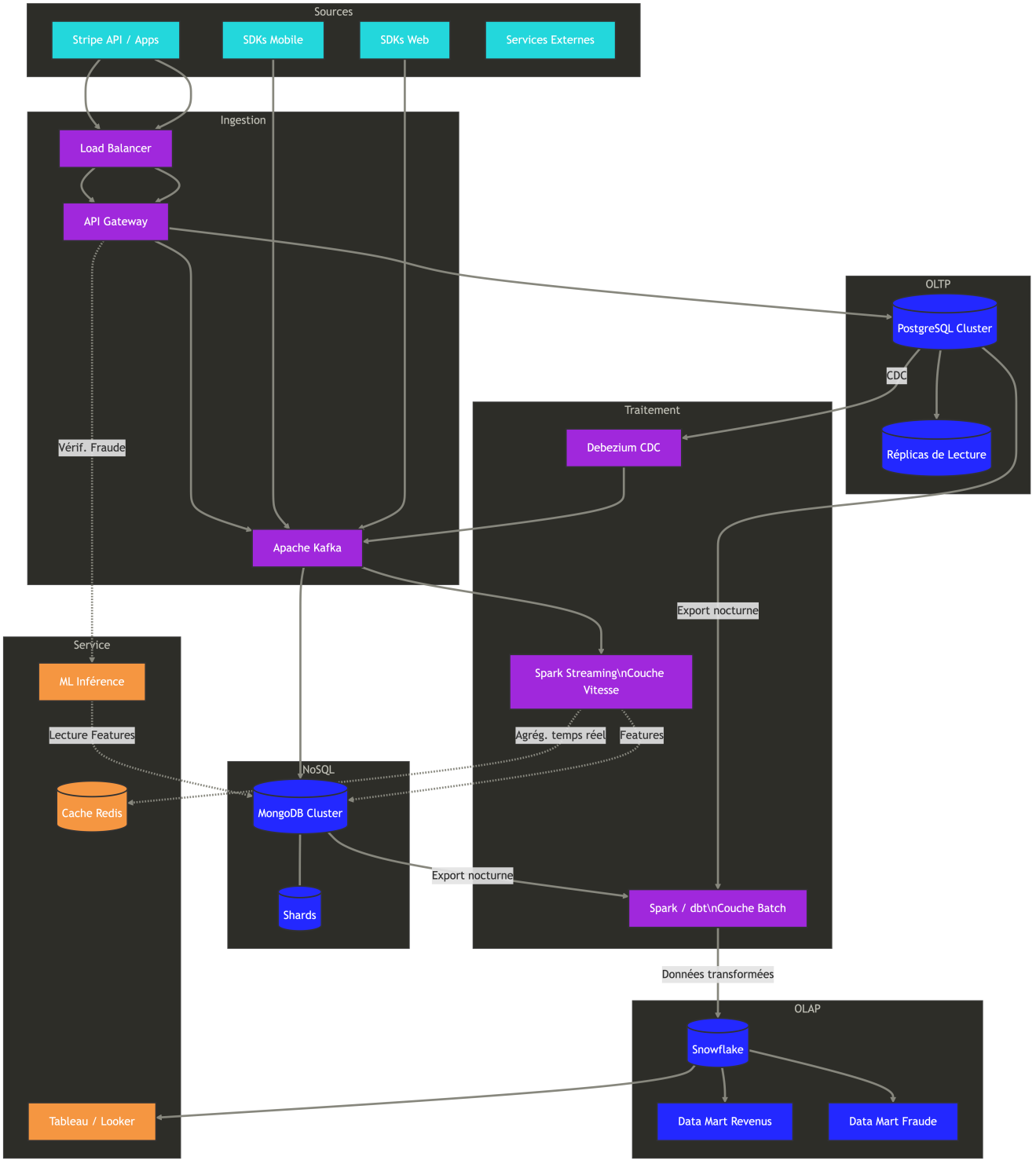
---

# LIVRABLE 1 - Diagramme d'Architecture de Données

L'architecture suit une approche Modern Data Stack combinée à un pattern Lambda Architecture : une couche vitesse pour les besoins temps réel (fraude, dashboards live) et une couche batch pour l'analytique lourde et le reporting réglementaire.

---

## 1. Architecture Cible




---

## 2. Flux de Données

L'architecture supporte quatre flux principaux :

- **Flux transactionnel (chemin chaud) :** Les applications écrivent dans PostgreSQL (OLTP) pour garantir la conformité ACID. Debezium capture les changements via CDC et les pousse vers Kafka, sans impacter les performances de la base transactionnelle.
- **Flux non structuré :** Les logs et événements des SDKs sont poussés directement vers Kafka, puis consommés et stockés dans MongoDB pour l'analyse et le Feature Store ML.
- **Flux analytique (chemin froid) :** Spark/dbt traite les données brutes depuis les archives et les charge dans Snowflake (OLAP). Les jointures complexes et agrégations alimentent les outils BI.
- **Flux temps réel (couche vitesse) :** Spark Streaming consomme depuis Kafka pour calculer des fenêtres glissantes. Les modèles ML utilisent les features MongoDB/Redis pour scorer les transactions en millisecondes.

---

## 3. Pipelines

### Couche Vitesse (SLA < 10s)

**Technologies :** Flink / Spark Streaming

Le pipeline ingère depuis Kafka, applique des transformations légères (filtrage, agrégations fenêtrées), enrichit avec le cache Redis (profils utilisateur), puis écrit vers Redis (compteurs live) et MongoDB (Feature Store ML).

### Couche Batch (SLA < 15 min)

**Technologies :** Airflow (orchestration) + dbt (transformation) + Snowflake (calcul)

Processus ELT en 4 étapes :

1. **Extract :** Airflow déclenche l'extraction depuis les archives Kafka (S3) et les snapshots DB
2. **Load :** Chargement des données brutes dans le schéma `RAW` de Snowflake
3. **Transform :** dbt exécute les modèles SQL (nettoyage, jointures OLTP/NoSQL, modèles dimensionnels)
4. **Test :** dbt vérifie la qualité des données (unicité, intégrité référentielle)

---

## 4. Intégration Inter-systèmes

| Chemin | Méthode | Outils | Fréquence |
|---|---|---|---|
| OLTP → NoSQL | Événementiel | Kafka + Connecteurs | Temps réel |
| OLTP → OLAP | ELT / CDC | Debezium + Snowflake | < 5 min |
| NoSQL → OLAP | Batch | Airflow + Snowflake | Horaire / Quotidien |
| OLAP → NoSQL | Reverse ETL | HighTouch / Script | Quotidien |

---

## 5. Choix des Outils

- **Apache Kafka :** Standard pour le streaming d'événements à haut débit. Découple producteurs et consommateurs. Chaque système évolue indépendamment.
- **Debezium :** Outil open-source de référence pour le CDC sur PostgreSQL. Capture chaque état intermédiaire (INSERT, UPDATE) via le WAL, garantissant une auditabilité totale sans impacter les performances de la base source.
- **dbt :** Permet de gérer la logique métier SQL comme du code (Git, tests, CI/CD), garantissant des définitions de KPIs versionnées et cohérentes.
- **Snowflake :** Séparation indépendante du stockage et du calcul. Permet de monter des clusters pour les rapports mensuels puis de les éteindre (maîtrise des coûts).

---

## 6. Disaster Recovery (DR)

| Système | RPO | RTO | Mécanisme |
|---|---|---|---|
| *OLTP (PostgreSQL)* | < 1 min | < 5 min | Streaming replication + failover automatique (Patroni) |
| *NoSQL (MongoDB)* | < 1 min | < 10 min | Replica set (3 nœuds min.) + élection automatique |
| *OLAP (Snowflake)* | 0 | < 1 h | Time Travel (90 jours) + Fail-safe (7 jours) |
| *Kafka* | 0 | < 5 min | Replication factor 3 + controller failover |

### Stratégie de sauvegarde

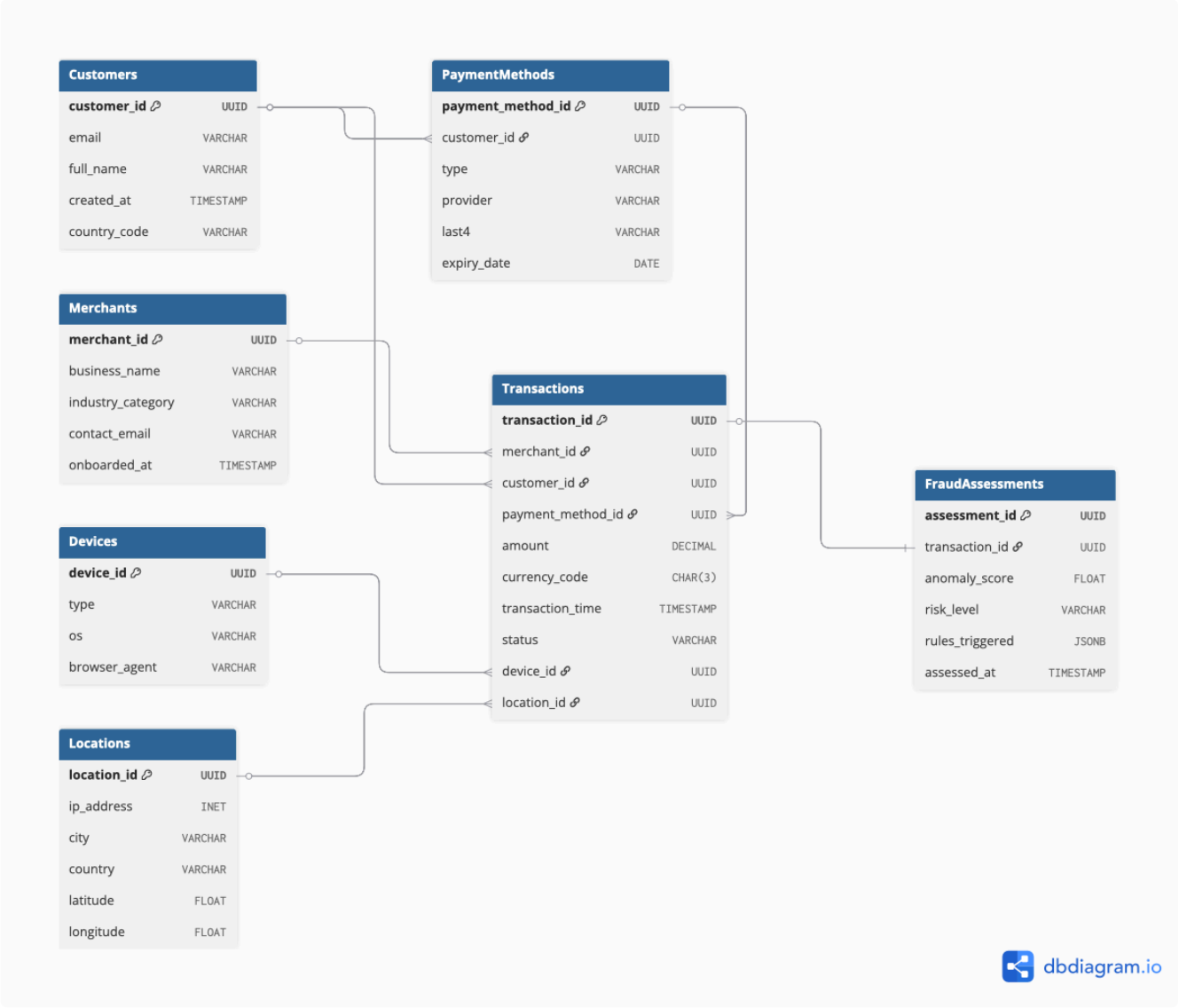
- **Snapshots quotidiens automatisés** stockés en cross-région (S3 Cross-Region Replication) pour survivre à une panne régionale complète.
- **Tests de restauration trimestriels** pour valider l'intégrité des sauvegardes et les procédures de bascule.
- **Runbooks documentés** pour chaque scénario de panne (perte d'un nœud, perte d'une zone, perte d'une région).

# LIVRABLE 2 - Diagramme ERD - Modèle OLTP

Ce schéma est conçu pour le système transactionnel principal de Stripe. Il repose sur une normalisation en 3NF pour garantir l'intégrité des données (ACID), réduire la redondance et optimiser les performances en écriture sur de gros volumes.

---

## Diagramme Entité-Relation




---

## Décisions de Conception

**Normalisation 3NF :** Les tables `Devices` et `Locations` sont séparées de `Transactions` pour éviter la duplication massive de chaînes de caractères (ex : "Chrome on MacOS" répété des millions de fois). De même, `PaymentMethods` est séparé de `Customers` pour permettre plusieurs méthodes par client sans dupliquer les informations personnelles.

**Types de données :** Les UUIDs sont utilisés pour toutes les clés primaires afin de garantir l'unicité globale en environnement distribué (essentiel pour le sharding futur). Le type `DECIMAL` est imposé pour les montants financiers — les flottants introduiraient des erreurs d'arrondi inacceptables. Le type `INET` est le type natif PostgreSQL pour les adresses IP.

**Fraude séparée :** `FraudAssessments` est une table dédiée (relation 1:1 optionnelle) pour permettre un traitement asynchrone de l'analyse de risque sans verrouiller la transaction principale, et pour stocker les résultats complexes (règles déclenchées en JSONB) séparément.

### Indexation

| Index | Colonne(s) | Usage |
|---|---|---|
| `idx_tx_merchant_time` | `(merchant_id, transaction_time)` | Dashboards marchands |
| `idx_tx_customer` | `(customer_id)` | Historique client |
| `idx_tx_status` | `(status)` | Monitoring opérationnel |
| `idx_pending_tx` | `(status) WHERE status='Pending'` | File fraude (index partiel) |

---

## Requêtes de Validation

**Q1 — Historique client** (app mobile)

```
SELECT t.transaction_id, m.business_name, t.amount, t.currency_code, t.status, t.trans
FROM Transactions t
JOIN Merchants m ON t.merchant_id = m.merchant_id
WHERE t.customer_id = 'cst_12345'
ORDER BY t.transaction_time DESC LIMIT 20;
```

**Q2 — Transactions haut risque** (analystes fraude)

```
SELECT t.transaction_id, t.amount, c.email, f.risk_level, f.anomaly_score
FROM Transactions t
JOIN FraudAssessments f ON t.transaction_id = f.transaction_id
JOIN Customers c ON t.customer_id = c.customer_id
WHERE f.risk_level IN ('High', 'Critical') AND t.status = 'Pending'
ORDER BY f.anomaly_score DESC;
```

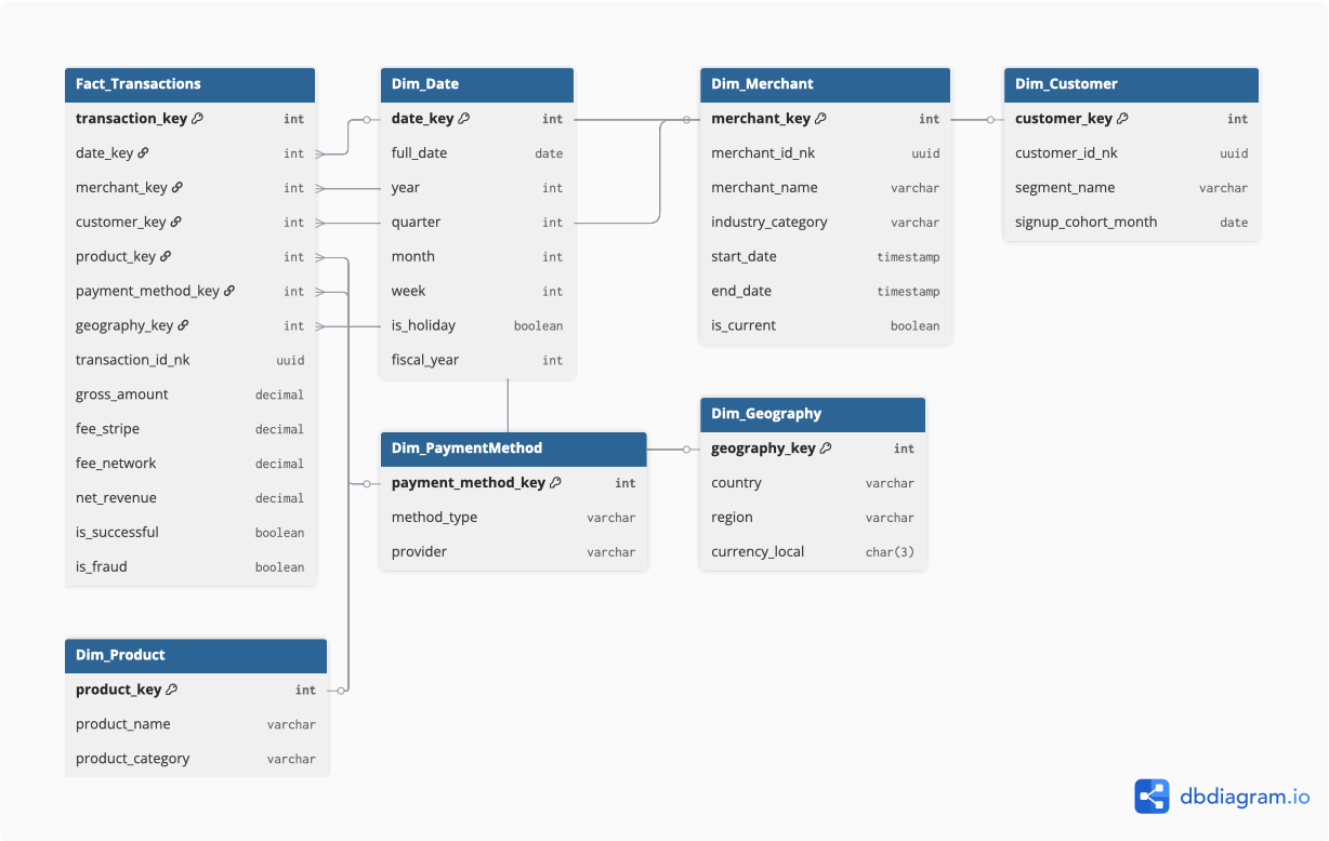
---

# LIVRABLE 3 - Diagramme OLAP - Schéma Analytique

L'entrepôt de données utilise un schéma en étoile (Star Schema), choisi pour sa performance en requêtage et sa lisibilité par les outils BI et les utilisateurs métier.

---

## Schéma de type Étoile




## Vues Matérialisées (Pré-agrégation)

Pour garantir des temps de réponse inférieurs à la seconde sur les dashboards, deux vues matérialisées pré-calculent les métriques les plus demandées :

| Vue | Group By | Métriques | Usage |
|---|---|---|---|
| `Agg_Daily_Revenue_Merch` | `date_key`, `merchant_key` | SUM(gross_amount), SUM(net_revenue), COUNT(*) | Dashboards marchands |
| `Agg_Monthly_Global_Perf` | `month`, `geography_key`, `industry_category` | SUM(gross_amount), AVG(fee_stripe) | Reporting exécutif |

---

## Décisions de Conception

**Clés substituts (Surrogate Keys) :** Les jointures utilisent des entiers (`_key`) au lieu des UUIDs de l'OLTP pour améliorer les performances dans le stockage colonnaire. Les UUIDs sont conservés comme clés naturelles (`_nk`) pour la traçabilité vers le système source.

**SCD Type 2 sur Dim_Merchant :** La dimension Marchand utilise le pattern Slowly Changing Dimension Type 2 pour conserver l'historique des changements (nom commercial, catégorie). Si un marchand change de catégorie, ses anciennes transactions restent liées à l'ancienne catégorie dans les rapports historiques.

**Dim_Product :** La dimension Produit segmente les transactions par ligne de produit Stripe (Payments, Billing, Connect, Terminal). Bien que la cardinalité soit faible, cette dimension est essentielle pour l'analyse croisée du revenu par produit et par marchand.

**Dim_PaymentMethod :** La dimension Méthode de Paiement permet d'analyser les revenus par type (carte, virement, API) et par fournisseur (Visa, MasterCard, PayPal). C'est un axe d'analyse métier clé pour comprendre les préférences de paiement et optimiser les coûts réseau.

**Pré-agrégation :** Les vues matérialisées évitent de rescanner des milliards de lignes à chaque actualisation de dashboard, répondant à l'exigence de métriques de revenus instantanées.

---

## Requêtes de Validation

**Q1 — Rapport revenus quotidien** (BI)

```
SELECT d.full_date, g.region,
    SUM(f.gross_amount) AS total_brut, SUM(f.net_revenue) AS total_net, COUNT(*) AS nb
FROM Fact_Transactions f
JOIN Dim_Date d ON f.date_key = d.date_key
JOIN Dim_Geography g ON f.geography_key = g.geography_key
WHERE d.year = 2024 AND d.month = 1
GROUP BY 1, 2 ORDER BY 1 DESC, 3 DESC;
```

**Q2 — Analyse de cohortes** (stratégie produit)

```
SELECT c.signup_cohort_month, d.month AS mois_activite, SUM(f.gross_amount) AS depense
FROM Fact_Transactions f
JOIN Dim_Customer c ON f.customer_key = c.customer_key
JOIN Dim_Date d ON f.date_key = d.date_key
WHERE d.full_date >= c.signup_cohort_month
GROUP BY 1, 2 ORDER BY 1, 2;
```

---

# LIVRABLE 4 - Modèle de Données NoSQL

MongoDB est choisi comme base documentaire pour sa flexibilité avec les données JSON, sa scalabilité horizontale (sharding) et sa capacité à gérer des schémas évolutifs sans interruption de service. Il sert trois cas d'usage : stockage de logs, analyse comportementale et Feature Store ML.

---

## Collections

### 1. `ApplicationLogs`

Logs bruts pour le dépannage et l'audit. Un index TTL supprime automatiquement les données de plus de 30 jours, optimisant les coûts de stockage sans scripts de maintenance.

```
{
  "_id": "ObjectId",
  "timestamp": "ISODate",
  "level": "ERROR",
  "service_name": "payment-gateway",
  "message": "Timeout de connexion",
  "metadata": { "request_id": "uuid", "stack_trace": "..." },
  "expire_at": "ISODate"
}
```

| Index | Champ(s) | Objectif |
|---|---|---|
| Temporel | `{ timestamp: -1 }` | Logs récents |
| Composite | `{ service_name: 1, level: 1 }` | Filtrage service/sévérité |
| TTL | `{ expire_at: 1 }` | Purge auto 30 jours |

### 2. `UserSessions` (Clickstream)

Agrège les actions utilisateur au sein d'une session. Les événements sont **embarqués** directement dans le document (pattern Embedding) pour garantir la localité des données : une seule lecture disque suffit pour récupérer tout le contexte d'une session.

```
{
  "_id": "session_uuid",
  "user_id": "customer_uuid",
  "start_time": "ISODate",
  "device_info": { "os": "iOS", "version": "15.0" },
  "events": [
    { "event_type": "view_item", "timestamp": "ISODate", "details": { "product_id": ">
    { "event_type": "add_to_cart", "timestamp": "ISODate", "details": { "product_id":
  ]
}
```

| Index | Champ(s) | Objectif |
|---|---|---|
| Historique | `{ user_id: 1, start_time: -1 }` | Recherche utilisateur |

## 3. `CustomerFeedback`

Avis et réponses aux enquêtes, liés aux transactions OLTP via `transaction_id`.

```
{
  "_id": "ObjectId",
  "customer_id": "customer_uuid",
  "transaction_id": "transaction_uuid",
  "rating": 5,
  "comment": "Excellent service !",
  "tags": ["rapide", "fiable"],
  "sentiment_score": 0.98
}
```

| Index | Champ(s) | Objectif |
|---|---|---|
| Transaction | `{ transaction_id: 1 }` | Lien OLTP |
| Tags | `{ tags: 1 }` | Index multi-clés |

## 4. `MLFeatures` (Feature Store)

Features pré-calculées garantissant la cohérence entre entraînement et inférence. Le schéma de l'objet `features` est volontairement flexible pour permettre aux data scientists d'ajouter de nouvelles features sans migration de base de données.

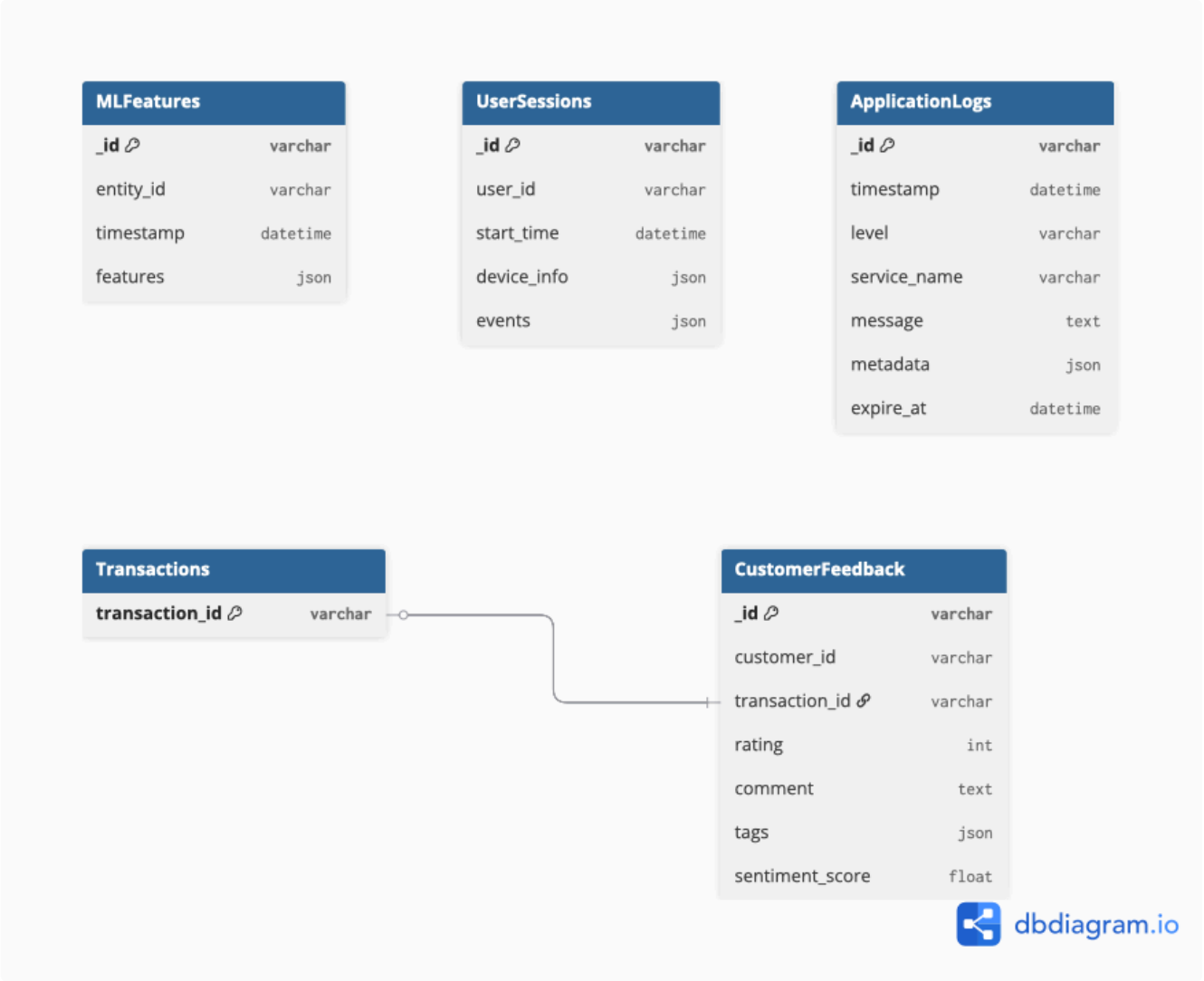
```
{
  "_id": "entity_id_timestamp_hash",
  "entity_id": "customer_uuid",
  "timestamp": "ISODate",
  "features": { "avg_spend_30d": 150.00, "tx_count_7d": 5, "last_login_country": "FR"
}
```

| Index | Champ(s) | Objectif |
|---|---|---|
| Lookup | `{ entity_id: 1, timestamp: -1 }` | Dernières features par entité |
| Shard Key | `{ entity_id: "hashed" }` | Distribution uniforme |
| TTL | `{ timestamp: 1 }` expireAfterSeconds: 180 jours | Purge des anciennes versions |

**Stratégie de sharding :** La shard key `{ entity_id: "hashed" }` garantit une distribution uniforme des documents sur les shards, évitant les hotspots liés aux clients très actifs. Combinée à un index TTL de 180 jours sur les anciennes versions de features, cette stratégie assure la scalabilité horizontale tout en maîtrisant les coûts de stockage.

---

## Schéma (approximation visuelle)




---

## Requêtes de Validation

### Q1 — Erreurs par service (DevOps)

```
db.ApplicationLogs.aggregate([
  { $match: { timestamp: { $gte: ISODate("2024-02-01T00:00:00Z") }, level: "ERROR" } }
  { $group: { _id: "$service_name", nb_erreurs: { $sum: 1 }, messages: { $push: "$mess
  { $sort: { nb_erreurs: -1 } }
]);
```

### Q2 — Vélocité utilisateur (feature ML fraude)

```
db.UserSessions.aggregate([
  { $match: { user_id: "cst_12345", start_time: { $gte: ISODate("2024-02-03T16:00:00Z"
  { $project: { tx: { $filter: { input: "$events", as: "e", cond: { $eq: ["$$e.event_t
  { $unwind: "$tx" },
  { $count: "tx_count_1h" }
]);
```

---

# LIVRABLE 5 - Sécurité et Conformité

En tant qu'entreprise FinTech traitant des données de paiement sensibles, Stripe doit garantir la sécurité des données à tous les niveaux et se conformer aux réglementations PCI-DSS, GDPR et CCPA.

---

## 1. Chiffrement

### Données au repos

| Système | Méthode |
|---|---|
| OLTP (PostgreSQL) | TDE/LUKS + AES-256 applicatif pour les PAN |
| NoSQL (MongoDB) | WiredTiger Encryption natif |
| OLAP (Snowflake) | Chiffrement par défaut |

### Données en transit

Toute communication interne et externe utilise TLS 1.3. Le mTLS (Mutual TLS) est imposé pour la communication service-à-service, notamment entre les producteurs/consommateurs Kafka.

### Gestion des clés

Un service KMS dédié (AWS KMS ou HashiCorp Vault) gère le cycle de vie des clés avec une rotation automatique tous les 90 jours.

### Chiffrement applicatif (PCI-DSS)

Le chiffrement disque seul ne suffit pas : un attaquant avec un accès root pourrait lire les fichiers déchiffrés. Les numéros de carte (PAN) sont donc chiffrés au niveau applicatif (AES-256) avant insertion dans la base. La DB ne stocke que des blobs chiffrés.

### Tokenisation

Pour l'analytique, les PAN sont remplacés par des tokens déterministes dès l'ingestion. Les analystes n'ont jamais besoin du numéro de carte réel pour calculer le chiffre d'affaires. L'entrepôt de données (OLAP) ne stocke jamais de PAN bruts.

---

## 2. Contrôle d'Accès (RBAC)

Chaque rôle reçoit le niveau d'accès minimum nécessaire (principe du moindre privilège) :

| Rôle | Accès | Périmètre |
|---|---|---|
| Service Applicatif | R/W | OLTP (tables spécifiques), Kafka (logs) |
| Data Engineer | R/W | Jobs ETL, Airflow, configs pipeline |
| Data Scientist | R | NoSQL (Features ML), OLAP (anonymisé) |
| Analyste BI | R | Data Marts OLAP (pré-agrégés) |
| Admin | Full | Infrastructure, gestion utilisateurs |

La mise en œuvre repose sur des rôles et groupes discrets dans PostgreSQL et MongoDB. Au niveau réseau, le VPC Peering et les Security Groups isolent les bases OLTP d'Internet.

---

## 3. Conformité Réglementaire

### PCI-DSS

| Exigence | Mise en œuvre |
|---|---|
| Isolation du périmètre | CDE (Cardholder Data Environment) dans une zone réseau restreinte |
| Tokenisation | PAN → tokens non sensibles pour l'analytique |
| Pistes d'audit | Journalisation stricte de tous les accès au CDE |

### GDPR / CCPA

| Droit | OLTP | OLAP | NoSQL |
|---|---|---|---|
| Accès | API par `customer_id` | Requêtes sur données anonymisées | Recherche par `customer_id` |
| Effacement | Suppression / anonymisation PII | Vecteurs de suppression ou rebuild batch | TTL auto + masquage PII dans les logs |
| Masquage | Dynamique sauf rôle Support | Pré-anonymisé à l'ingestion | Niveau applicatif |

---

## 4. Monitoring et Audit

Toute activité est tracée dans un système de journalisation centralisé : **qui** a accédé à **quelles** données et **quand**. Des alertes temps réel sont déclenchées pour les comportements suspects :

- **Connexions échouées multiples** → Tentative de force brute
- **Volume d'extraction inhabituel** → Risque d'exfiltration de données
- **Modification de schéma en production** → Altération non autorisée

### Outils de monitoring

| Outil | Rôle |
|---|---|
| **ELK Stack** (Elasticsearch, Logstash, Kibana) | Agrégation centralisée des logs et recherche full-text |
| **Prometheus + Grafana** | Métriques d'infrastructure et applicatives (latence, saturation, erreurs) |
| **SIEM** (Splunk ou équivalent) | Corrélation d'événements de sécurité et détection d'incidents |
| **PagerDuty / OpsGenie** | Routage des alertes et gestion des incidents (on-call) |

## SLA de réponse aux incidents

| Priorité | Description | Temps de réponse | Temps de résolution |
|---|---|---|---|
| P1 | Fuite de données, exfiltration avérée | < 15 min | < 4 h |
| P2 | Tentative d'accès non autorisé | < 1 h | < 24 h |
| P3 | Violation de politique interne | < 4 h | < 72 h |

# LIVRABLE 6 - Stratégie d'Intégration Machine Learning

Ce document décrit comment les modèles de Machine Learning s'intègrent dans l'architecture de données, principalement pour la **détection de fraude en temps réel** et la **personnalisation client**.

---

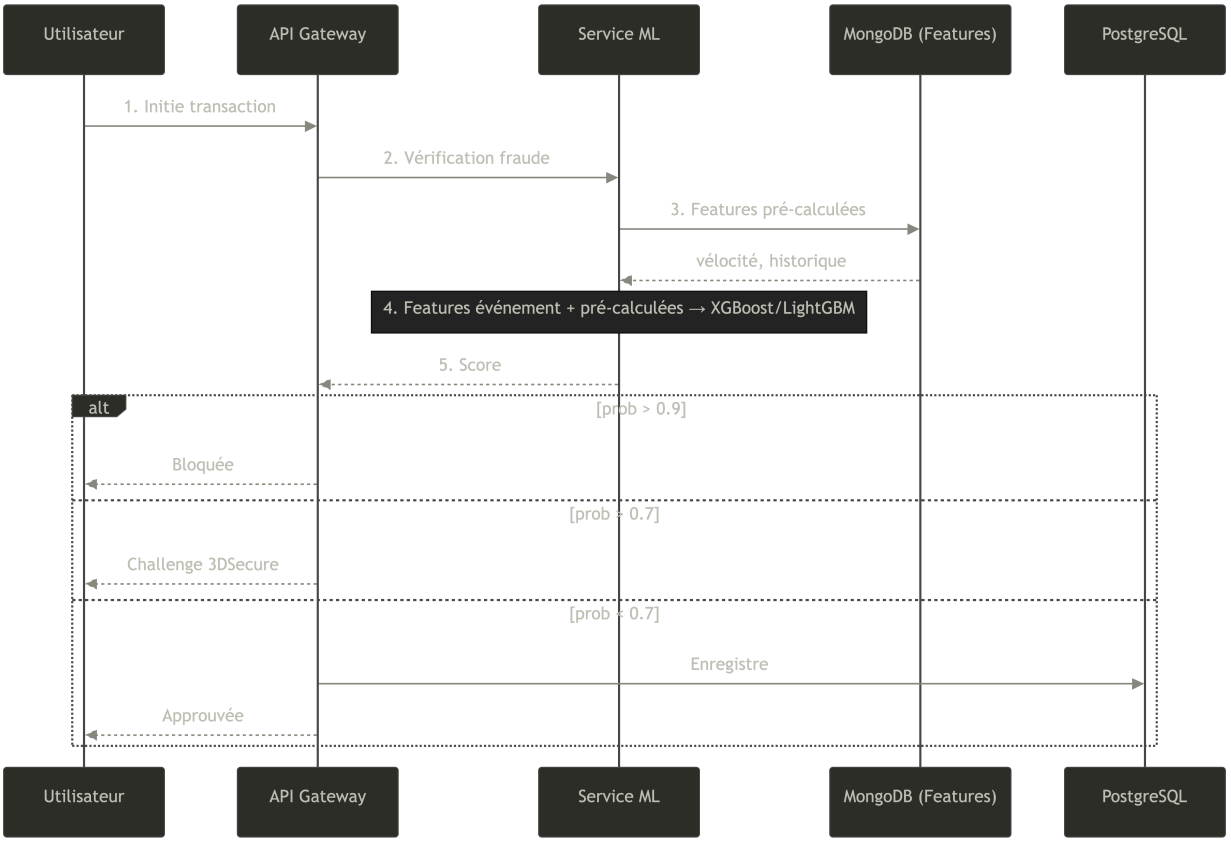
## 1. Feature Store

Le Feature Store résout le problème majeur du ML en production : les données d'entraînement (historiques) doivent correspondre exactement aux données de prédiction (temps réel). Il se compose de deux stores :

- **Online Store (MongoDB ou Redis) :** Valeurs actuelles des features pour l'inférence temps réel (ex : `last_1h_txn_count: 5`, `avg_spend_30d: 200.50`). MongoDB est choisi pour ses lectures clé-valeur rapides (ms) et son schéma flexible.
- **Offline Store (Snowflake) :** Valeurs historiques des features pour l'entraînement des modèles avec correction point-in-time.

---

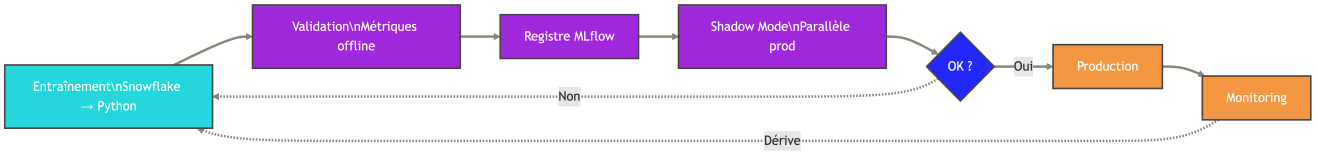
## 2. Détection de Fraude Temps Réel




L'ensemble du processus (étapes 1 à 5) doit s'exécuter en **< 100ms**. Le service ML récupère les features pré-calculées depuis MongoDB (vélocité utilisateur, historique de dépenses), les combine avec les features de l'événement en cours (montant, appareil), puis le modèle (XGBoost/LightGBM) prédit la probabilité de fraude.

---

## 3. MLOps — Cycle de Vie




- **Entraînement :** Déclenché de façon hebdomadaire ou par dérive de performance. Lecture des données historiques depuis Snowflake, entraînement en Python (Scikit-learn/PyTorch), enregistrement dans un Model Registry (MLflow).
- **Déploiement :** Le modèle est servi comme API REST conteneurisée (FastAPI) sur Kubernetes.
- **Shadow Mode :** Avant promotion en production, le nouveau modèle tourne en parallèle de l'existant sans bloquer les utilisateurs, pour valider ses performances sur du trafic réel.

---

## 4. Monitoring et Rétroaction

### Vérité terrain

La validation des prédictions repose sur deux sources retardées :

- **Rétrofacturations** (semaines) : Le client conteste la transaction — label retardé.
- **Revue manuelle** (jours) : Les analystes examinent les transactions signalées.

### Métriques surveillées

- **Prédictions vs réalité** : Comparaison des distributions de scores avec la vérité terrain (Prometheus + Grafana).
- **Dérive des features** : Détection de changements anormaux, par exemple un montant moyen qui double soudainement (Evidently AI).
- **Qualité des données** : Contrôles de fraîcheur et complétude des features (dbt tests + alertes).

---